



UNIVERSITÀ POLITECNICA DELLE MARCHE

Facoltà di Ingegneria

Corso di Laurea Triennale in Ingegneria Informatica e dell'Automazione

PROGETTO DI DATA SCIENCE

**Comparative Sentiment Analysis: Binary vs. Ternary
Classification on RateMyProfessors Data**

**Analisi Comparativa del Sentiment: Classificazione
Binaria e Ternaria sulle Recensioni di
RateMyProfessors**

Docente:

Prof. Domenico Ursino

Componenti del gruppo:

Meloccaro Lorenzo
Suarez Sanchez Yassir Flavio
La Porta Domenico

Anno Accademico 2025/2026

Indice

1	Introduzione al Natural Language Processing	3
1.1	Definizione e Rilevanza del NLP	3
1.2	Componenti Architettureali del NLP	3
1.3	Tecniche e Paradigmi Metodologici	4
2	Definizione del Problema e Dataset	4
2.1	Obiettivi del Progetto	4
2.1.1	Ipotesi di Ricerca	4
2.2	Dataset: RateMyProfessors Reviews	4
2.3	Analisi Esplorativa dei Dati	5
3	Metodologia	5
3.1	Architettura della Pipeline	5
3.2	Preprocessing del Testo	6
3.2.1	Pipeline di Preprocessing	6
3.2.2	Operazioni di Preprocessing Dettagliate	6
3.2.3	Esempio di Preprocessing End-to-End	8
3.3	Feature Extraction con TF-IDF	8
3.3.1	Configurazione del TF-IDF Vectorizer	8
3.3.2	Risultati della Vettorizzazione	9
3.4	Feature Engineering Custom	9
3.5	Gestione dello Sbilanciamento delle Classi	9
3.5.1	Strategie Testate	9
3.5.2	Valutazione delle Strategie	10
3.6	Modelli di Machine Learning	10
3.6.1	Modelli Base	10
3.6.2	Metodi Ensemble	11
3.7	Ottimizzazione degli Iperparametri	11
3.8	Validazione e Metriche	11
3.8.1	Split dei Dati	11
3.8.2	Cross-Validation	11
3.8.3	Metriche di Valutazione	12
3.9	Implementazione e Ambiente Tecnico	12
4	Esperimento 1: Classificazione Binaria	12
4.1	Distribuzione delle Classi	12
4.2	Risultati della Cross-Validation	13
4.3	Performance sul Test Set	13
4.3.1	Modello Migliore: Stacking Ensemble	15
4.3.2	Analisi della Confusion Matrix	15
4.3.3	Curve ROC e AUC	17
4.3.4	Curve Precision-Recall	17
4.4	Analisi delle Feature più Discriminanti	17
4.5	Curve di Apprendimento	17

5	Esperimento 2: Classificazione Multi-classe	18
5.1	Costruzione delle Etichette Ternarie	18
5.2	Distribuzione delle Classi	18
5.3	Risultati della Cross-Validation	21
5.4	Performance sul Test Set	21
5.4.1	Tutti i Modelli	21
5.4.2	Modello Migliore: Stacking Ensemble — Metriche per Classe	21
5.4.3	Analisi della Confusion Matrix	22
5.4.4	Curve ROC e AUC	22
6	Confronto tra Approcci	25
6.1	Sintesi dei Risultati	25
6.2	Analisi Comparativa	25
7	Conclusioni	27
7.1	Principali Risultati	27
7.2	Limitazioni	27
7.3	Sviluppi Futuri	27

1 Introduzione al Natural Language Processing

Il Natural Language Processing (NLP), o elaborazione del linguaggio naturale, rappresenta uno dei settori più dinamici e promettenti dell'intelligenza artificiale contemporanea. Questa disciplina si pone l'ambizioso obiettivo di permettere ai sistemi informatici di comprendere, interpretare e generare il linguaggio umano in forme sia scritte che parlate, colmando così il divario tra la comunicazione naturale dell'uomo e la logica computazionale delle macchine.

L'importanza del NLP nella società moderna è testimoniata dalla pervasività delle sue applicazioni: dagli assistenti vocali che rispondono ai nostri comandi quotidiani, ai motori di ricerca che interpretano le nostre query, fino ai sofisticati sistemi di traduzione automatica che abbattano le barriere linguistiche. Secondo le stime di Grand View Research (2023), il mercato globale del NLP è destinato a raggiungere i 127,7 miliardi di dollari entro il 2028, con un tasso di crescita annuale composto del 29,4%, confermando la rilevanza strategica di questa tecnologia.

1.1 Definizione e Rilevanza del NLP

Il Natural Language Processing può essere formalmente definito come l'insieme di tecniche computazionali volte all'analisi e alla sintesi del linguaggio umano naturale. A differenza dei linguaggi formali utilizzati in informatica, il linguaggio naturale è caratterizzato da ambiguità semantica, variabilità contestuale e ricchezza espressiva, proprietà che rendono la sua elaborazione automatica particolarmente complessa.

Definizione Formale

Il Natural Language Processing (NLP) è un campo interdisciplinare che combina linguistica computazionale, intelligenza artificiale e apprendimento automatico per consentire l'interazione tra computer e linguaggio umano in modo significativo e utile [1].

La rilevanza del NLP nella società contemporanea può essere analizzata secondo tre dimensioni principali:

- **Dimensione tecnologica:** il NLP costituisce l'infrastruttura abilitante per tecnologie ormai ubiquie come la ricerca semantica, l'assistenza virtuale, la traduzione automatica e l'analisi del sentiment nei social media.
- **Dimensione economica:** l'automazione di processi basati sul linguaggio (customer service, analisi documentale, data mining testuale) genera significativi risparmi di costo e incrementi di efficienza per le organizzazioni.
- **Dimensione sociale:** le tecnologie NLP stanno democratizzando l'accesso all'informazione attraverso sistemi di ricerca più intuitivi, assistenti per persone con disabilità e strumenti di traduzione che facilitano la comunicazione interculturale.

1.2 Componenti Architettureali del NLP

L'architettura del Natural Language Processing si articola in tre componenti fondamentali: Natural Language Understanding (NLU), Natural Language Generation (NLG) e

Core Processing. Ciascuna componente affronta specifiche problematiche computazionali e contribuisce all’obiettivo complessivo di enabling human-computer linguistic interaction.

1.3 Tecniche e Paradigmi Metodologici

L’evoluzione del Natural Language Processing è stata caratterizzata da un progressivo passaggio da approcci rule-based a metodologie data-driven, fino all’attuale dominanza di tecniche basate su deep learning.

Gli approcci statistici al NLP si basano sull’idea di inferire pattern linguistici da grandi corpora attraverso modelli probabilistici. Il TF-IDF (Term Frequency-Inverse Document Frequency), utilizzato estensivamente nel presente lavoro, calcola per ogni termine t in un documento d appartenente a un corpus D :

$$\text{TF-IDF}(t, d, D) = \text{TF}(t, d) \times \text{IDF}(t, D) \quad (1)$$

dove:

$$\text{TF}(t, d) = \frac{f_{t,d}}{\sum_{t' \in d} f_{t',d}} \quad \text{IDF}(t, D) = \log \frac{|D|}{|\{d \in D : t \in d\}|} \quad (2)$$

Nel presente lavoro verranno impiegati estensivamente metodi di machine learning tradizionale (SVM, Logistic Regression, Random Forest, Naive Bayes), integrati con TF-IDF e sampling strategies. La scelta di utilizzare approcci classici permette interpretabilità dei risultati ed efficienza computazionale.

2 Definizione del Problema e Dataset

2.1 Obiettivi del Progetto

L’obiettivo principale di questo studio è confrontare due approcci alternativi alla sentiment analysis:

1. **Classificazione Binaria:** distinzione tra recensioni positive e negative, utilizzando le etichette originali del dataset.
2. **Classificazione Multi-classe:** estensione a tre categorie (positivo, neutro, negativo) mediante generazione automatica della classe intermedia.

2.1.1 Ipotesi di Ricerca

- **H1:** La classificazione binaria raggiungerà performance metriche superiori (F1-score > 0.90).
- **H2:** L’introduzione della classe neutra offrirà maggiore realismo applicativo.
- **H3:** Tecniche di bilanciamento miglioreranno significativamente le performance.

2.2 Dataset: RateMyProfessors Reviews

Il dataset utilizzato proviene dalla piattaforma RateMyProfessors, disponibile su HuggingFace Datasets Hub.

Caratteristica	Valore
Numero totale recensioni	39.522
Lingua	Inglese
Etichette originali	2 classi (positive, negative)
Distribuzione originale	73,8% positive, 26,2% negative
Imbalance ratio	2,81:1

Tabella 1: Caratteristiche principali del dataset RateMyProfessors

2.3 Analisi Esplorativa dei Dati

L'analisi preliminare evidenzia uno sbilanciamento significativo e una variabilità nella lunghezza delle recensioni (media: 258 caratteri, mediana: 300, range: 1-350). Le recensioni negative tendono ad essere leggermente più lunghe (271.8 caratteri medi) rispetto alle positive (253.2 caratteri).

3 Metodologia

Questo capitolo descrive in dettaglio la pipeline metodologica implementata, dalla pre-processing dei dati grezzi fino alla valutazione dei modelli. L'approccio adottato segue le best practices consolidate nella comunità NLP, integrandole con tecniche specifiche per affrontare le sfide del problema.

3.1 Architettura della Pipeline

La pipeline di sentiment analysis implementata si articola in cinque macro-fasi sequenziali:

1. **Preprocessing e Pulizia:** normalizzazione del testo, rimozione noise, tokenizzazione.
2. **Feature Extraction:** trasformazione TF-IDF e feature engineering custom.
3. **Gestione Sbilanciamento:** applicazione di tecniche di bilanciamento delle classi.
4. **Training e Tuning:** addestramento modelli con ottimizzazione iperparametri.
5. **Valutazione:** cross-validation e test set evaluation con metriche multiple.

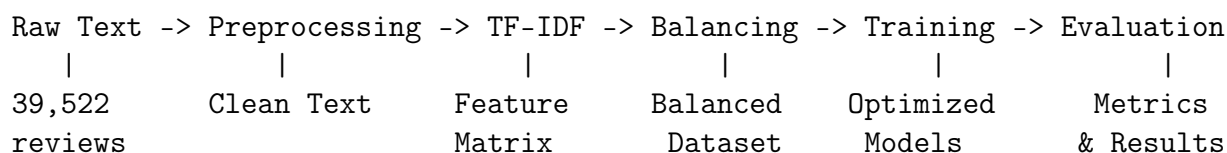


Figura 1: Schema generale della pipeline di sentiment analysis

3.2 Preprocessing del Testo

Il preprocessing costituisce una fase critica che influenza direttamente la qualità delle feature estratte e, di conseguenza, le performance dei modelli.

3.2.1 Pipeline di Preprocessing

La pipeline di preprocessing implementata segue questa sequenza di operazioni:

Algorithm 1 Pipeline di Preprocessing Testuale

```
1: procedure PREPROCESSTEXT(text)
2:   text  $\leftarrow$  Lowercase(text)
3:   text  $\leftarrow$  RemoveURLs(text)
4:   text  $\leftarrow$  RemovePunctuation(text)
5:   text  $\leftarrow$  RemoveNumbers(text)
6:   tokens  $\leftarrow$  Tokenize(text)
7:   tokens  $\leftarrow$  RemoveStopwords(tokens)
8:   tokens  $\leftarrow$  HandleNegations(tokens)
9:   tokens  $\leftarrow$  Lemmatize(tokens)
10:  tokens  $\leftarrow$  FilterShortTokens(tokens, min_length = 3)
11:  return JoinTokens(tokens)
12: end procedure
```

3.2.2 Operazioni di Preprocessing Dettagliate

Normalizzazione del Testo

- **Lowercasing:** conversione di tutti i caratteri in minuscolo per ridurre la dimensionalità del vocabolario. Ad esempio, “Professor”, “professor” e “PROFESSOR” vengono unificati in un’unica forma.
- **Rimozione URL:** eliminazione di pattern del tipo `http://`, `https://`, `www.` mediante espressioni regolari.
- **Rimozione Punteggiatura:** eliminazione di caratteri di punteggiatura che generalmente non contribuiscono al sentiment (ad eccezione di esclamativi e interrogativi che vengono conteggiati come feature).
- **Rimozione Numeri:** eliminazione di cifre numeriche che raramente portano informazione semantica rilevante per il sentiment.

Tokenizzazione La tokenizzazione segmenta il testo continuo in unità discrete (token). Utilizziamo il tokenizer di NLTK che gestisce correttamente:

- Contrazioni inglesi (“don’t” $\rightarrow$ “do”, “n’t”)
- Possessivi (“professor’s” $\rightarrow$ “professor”, “s”)
- Abbreviazioni comuni (“Dr.”, “Prof.”)

Rimozione Stop Words Le stop words sono parole ad alta frequenza ma basso contenuto informativo (articoli, preposizioni, pronomi). Utilizziamo la lista standard di NLTK per l'inglese, contenente 179 parole quali “the”, “is”, “at”, “which”, “on”.

La rimozione delle stop words riduce la dimensionalità del vocabolario del 30-40% senza perdita significativa di informazione per task di sentiment analysis.

Gestione delle Negazioni Un aspetto critico del preprocessing per sentiment analysis è la gestione delle negazioni, che invertono la polarità del sentiment. Implementiamo una strategia di negation handling che:

1. Identifica parole di negazione: {“not”, “no”, “never”, “neither”, “nobody”, “nothing”, “n’t”}
2. Prefixa con “NOT_” le parole immediatamente successive alla negazione:
 - “not good” → “not NOT_good”
 - “never satisfied” → “never NOT_satisfied”

Questa tecnica permette al modello di distinguere semanticamente “good” da “NOT_good”, migliorando la capacità di catturare sentiment negativo espresso mediante negazione di termini positivi.

Esempio di Negation Handling:

Input: “The professor is not helpful and never explains clearly”

Dopo negation handling: “professor NOT_helpful NOT_explains clearly”

Questa trasformazione permette al TF-IDF di trattare “NOT_helpful” come un termine distinto con alta correlazione con sentiment negativo.

Lemmatizzazione La lemmatizzazione riduce le parole alle loro forme base (lemmi) considerando il contesto morfologico. Utilizziamo WordNetLemmatizer di NLTK che mappa:

- Forme verbali: “teaches”, “teaching”, “taught” → “teach”
- Plurali: “professors”, “classes” → “professor”, “class”
- Comparativi: “better”, “best” → “good”

La lemmatizzazione è preferita allo stemming (che applica regole euristiche più aggressive) perché produce sempre parole valide del vocabolario, migliorando l’interpretabilità.

Filtering e Pulizia Finale

- **Rimozione token corti:** eliminazione di token con lunghezza < 3 caratteri per ridurre noise.
- **Rimozione testi vuoti:** esclusione di recensioni che dopo preprocessing risultano vuote (52 recensioni, 0.13% del dataset).

Testo Originale	Testo Processato
“Dr. Smith is an AMAZING professor! She explains complex topics very clearly and is always helpful during office hours.”	“smith amazing professor explain complex topic clearly helpful office hour”
“The professor doesn’t explain anything well. Tests are unfair and grading is inconsistent.”	“professor NOT_explain anything well test unfair grading inconsistent”

Tabella 2: Esempi di trasformazione mediante preprocessing

3.2.3 Esempio di Preprocessing End-to-End

3.3 Feature Extraction con TF-IDF

Dopo il preprocessing, il testo viene trasformato in rappresentazioni numeriche mediante TF-IDF vectorization.

3.3.1 Configurazione del TF-IDF Vectorizer

Parametri TF-IDF:

- **max_features = 5000:** limita il vocabolario alle 5000 feature più discriminanti
- **ngram_range = (1, 2):** include sia unigrammi che bigrammi
- **min_df = 3:** ignora termini che appaiono in meno di 3 documenti
- **max_df = 0.85:** ignora termini che appaiono in oltre l’85% dei documenti
- **sublinear_tf = True:** applica scaling logaritmico alle frequenze

Motivazione delle Scelte

- **5000 features:** bilanciamento tra ricchezza informativa e efficienza computazionale. Test preliminari hanno mostrato rendimenti decrescenti oltre questo limite.
- **Bigrams:** cattura espressioni composte rilevanti per il sentiment (“very good”, “not bad”, “highly recommend”) che unigrammi isolati non catturerebbero.
- **min_df = 3:** filtra typo, nomi propri e termini idiosincratici che non generalizzano.
- **max_df = 0.85:** rimuove termini ubiqui che aggiungono poco potere discriminativo (simile a stop words ma data-driven).
- **sublinear_tf:** previene che documenti con alta ripetizione di pochi termini dominino la rappresentazione. Formula: $TF_{\text{sub}} = 1 + \log(TF)$.

3.3.2 Risultati della Vettorizzazione

Applicando il TF-IDF al dataset preprocessato otteniamo:

- **Dimensione matrice:** (39522, 5000) (39.522 documenti $\times$ 5.000 feature)
- **Sparsità:** 99.2% (la maggioranza degli elementi è zero)
- **Vocabolario effettivo:** 5.000 termini selezionati
- **Formato:** Sparse matrix (scipy.sparse.csr_matrix) per efficienza memoria

3.4 Feature Engineering Custom

Oltre al TF-IDF, vengono estratte feature numeriche custom che catturano caratteristiche stilistiche e strutturali del testo:

Feature	Descrizione
text_length	Lunghezza in caratteri del testo originale
word_count	Numero di parole nel testo originale
avg_word_length	Lunghezza media delle parole
exclamation_count	Numero di punti esclamativi (indicatore di enfasi)
question_count	Numero di punti interrogativi
uppercase_ratio	Rapporto tra caratteri maiuscoli e totali
polarity	Polarity score di TextBlob (solo per versione 3 classi)
contrast_words	Conta di parole contrastive (“but”, “however”, “although”)

Tabella 3: Feature custom estratte dai testi

Queste feature catturano segnali paralinguistici che possono correlare con il sentiment ma che il TF-IDF non codifica esplicitamente.

3.5 Gestione dello Sbilanciamento delle Classi

Lo sbilanciamento iniziale (73,8% positive vs 26,2% negative) richiede strategie di mitigazione per evitare che i modelli sviluppino bias verso la classe maggioritaria.

3.5.1 Strategie Testate

Sono state confrontate le seguenti strategie di bilanciamento:

1. **No Sampling (Baseline):** nessuna modifica, training su dati originali sbilanciati.
2. **SMOTE (Synthetic Minority Over-sampling Technique):** genera esempi sintetici della classe minoritaria interpolando tra istanze esistenti nello spazio delle feature.
3. **ADASYN (Adaptive Synthetic Sampling):** variante di SMOTE che genera più esempi sintetici nelle regioni difficili da classificare.

4. **Random Oversampling:** duplicazione casuale di istanze della classe minoritaria fino a bilanciamento.

3.5.2 Valutazione delle Strategie

Per ciascuna strategia, si addestra un Logistic Regression e si valuta l’F1-score sul test set. I risultati ottenuti dall’esecuzione sperimentale della pipeline sono riportati in Tabella 4.

Strategia	Accuracy	F1-Score (weighted)
No Sampling	0.9293	0.9281
SMOTE	0.9285	0.9276
ADASYN	0.9267	0.9254
Random Oversampling	0.9299	0.9289

Tabella 4: Confronto strategie di bilanciamento (probe con Logistic Regression, classificazione binaria)

Risultato: Il confronto empirico ha mostrato differenze contenute tra le strategie: il Random Oversampling ottiene il miglior F1-score (0,9289) e viene selezionato per la classificazione binaria. Vale la pena notare che anche la baseline senza sampling performa in modo competitivo ($F1 = 0,9281$), suggerendo che l’imbalance ratio di 2,81:1 non costituisce una criticità grave per i modelli lineari su spazi TF-IDF. Per la classificazione a tre classi, si utilizzano class weights differenziati (**Negative:** 3,0, **Neutral:** 2,0, **Positive:** 1,0) per compensare le differenze di rappresentazione senza modificare la distribuzione dei dati di training.

3.6 Modelli di Machine Learning

Sono stati implementati e valutati sei modelli di classificazione:

3.6.1 Modelli Base

Naive Bayes (MultinomialNB) Classificatore probabilistico basato sul teorema di Bayes con assunzione di indipendenza condizionale. Particolarmente efficace con feature bag-of-words/TF-IDF.

Iperparametro principale: α (smoothing), testato in $\{0.1, 0.5, 1.0\}$.

Logistic Regression Modello lineare che stima probabilità mediante funzione logistica:

$$P(y = 1|x) = \frac{1}{1 + e^{-(\beta_0 + \beta^T x)}}$$

Iperparametri: C (inverso di regolarizzazione) $\in \{0.1, 1.0, 10.0\}$, `class_weight` $\in \{\text{None}, \text{“balanced”}\}$.

Support Vector Machine (SVM) Cerca l’iperpiano ottimale che massimizza il margine tra classi. Con kernel lineare, efficace in spazi ad alta dimensionalità come TF-IDF.

Iperparametri: $C \in \{0.1, 1.0, 10.0\}$, `kernel` = “linear”.

Random Forest Ensemble di decision trees addestrati su bootstrap samples. Riduce overfitting tramite averaging.

Iperparametri: `n_estimators` $\in \{100, 200\}$, `max_depth` $\in \{\text{None}, 20\}$, `min_samples_split` $\in \{2, 5\}$.

3.6.2 Metodi Ensemble

Voting Ensemble (Soft) Combina predizioni di modelli base mediante media delle probabilità:

$$\hat{y} = \arg \max_c \sum_{i=1}^M P_i(y = c|x)$$

Voting Ensemble (Hard) Voto a maggioranza tra predizioni discrete dei modelli base.

Stacking Ensemble Meta-modello (Logistic Regression) addestrato sulle predizioni dei modelli base come feature. Consente al meta-learner di apprendere come pesare i singoli classificatori in funzione delle loro aree di forza e debolezza.

3.7 Ottimizzazione degli Iperparametri

Gli iperparametri vengono ottimizzati mediante `RandomizedSearchCV` con:

- **Scoring:** F1-score (weighted per multi-classe)
- **Cross-validation:** 3-fold stratified
- **Iterazioni:** 10 combinazioni casuali per modello
- **Parallelizzazione:** `n_jobs` = -1 (tutti i core)

L'utilizzo di `RandomizedSearchCV` al posto di `GridSearchCV` riduce significativamente il costo computazionale mantenendo una buona copertura dello spazio degli iperparametri.

3.8 Validazione e Metriche

3.8.1 Split dei Dati

- **Training + Validation:** 80% (31.617 recensioni)
- **Test Set:** 20% (7.905 recensioni)
- **Stratificazione:** preserva proporzioni delle classi in train/test
- **Random seed:** 42 (per riproducibilità)

3.8.2 Cross-Validation

Stratified 5-Fold Cross-Validation sul training set per stima robusta delle performance:

$$\text{CV-Score} = \frac{1}{5} \sum_{i=1}^5 \text{F1-score}_i$$

3.8.3 Metriche di Valutazione

Classificazione Binaria

- Accuracy, Precision, Recall, F1-score
- AUC-ROC
- Confusion Matrix

Classificazione Multi-classe

- Weighted averages di Precision, Recall, F1
- Confusion Matrix 3×3
- Per-class metrics
- AUC-ROC one-vs-rest per ciascuna classe

3.9 Implementazione e Ambiente Tecnico

Stack Tecnologico:

- **Linguaggio:** Python 3.12
- **ML Framework:** scikit-learn 1.3+
- **NLP:** NLTK, TextBlob
- **Gradient Boosting:** XGBoost, LightGBM
- **Sampling:** imbalanced-learn
- **Data:** pandas, numpy
- **Ambiente:** Jupyter Notebook + script .py standalone

La pipeline completa è implementata in modo modulare e riproducibile, con configurazioni centralizzate e logging dettagliato di tutte le fasi.

4 Esperimento 1: Classificazione Binaria

Questo capitolo presenta i risultati dell'esperimento di classificazione binaria, in cui l'obiettivo è distinguere recensioni positive da negative utilizzando le etichette originali del dataset RateMyProfessors.

4.1 Distribuzione delle Classi

Il dataset originale presenta uno sbilanciamento significativo tra le due classi: 23.324 recensioni positive (73,8%) e 8.293 negative (26,2%) nel training set, per un totale di 39.522 campioni validi dopo il preprocessing. L'imbalance ratio risultante è di circa 2,81:1.

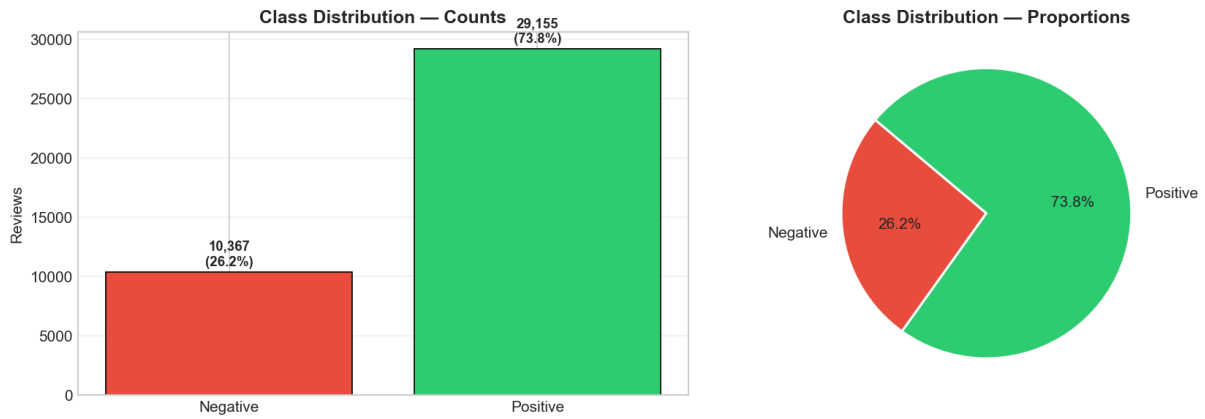


Figura 2: Distribuzione delle classi nel dataset — Classificazione Binaria

Come descritto nella sezione metodologica, tale sbilanciamento è stato mitigato mediante Random Oversampling, che ha dimostrato empiricamente le performance migliori tra le strategie testate.

4.2 Risultati della Cross-Validation

La Tabella 5 riporta i risultati della 5-Fold Stratified Cross-Validation sul training set per i sei modelli base considerati.

Modello	F1-Score (weighted)	Std
Linear SVM	0.9321	± 0.0027
Logistic Regression	0.9307	± 0.0026
LightGBM	0.9254	± 0.0022
Naive Bayes	0.9202	± 0.0034
XGBoost	0.9175	± 0.0013
Random Forest	0.9038	± 0.0015

Tabella 5: Risultati della 5-Fold Stratified Cross-Validation — Classificazione Binaria

I modelli lineari (Linear SVM e Logistic Regression) si collocano in cima alla classifica con F1-score rispettivamente di 0,9321 e 0,9307, confermando la loro efficacia in spazi ad alta dimensionalità come quello generato dalla vettorizzazione TF-IDF. LightGBM e XGBoost, pur performando adeguatamente (0,9254 e 0,9175), mostrano un leggero svantaggio rispetto ai modelli lineari, probabilmente dovuto alla difficoltà degli alberi decisionali nel lavorare efficientemente con matrici sparse ad alta dimensionalità. Random Forest risulta il modello meno performante (0,9038), sebbene comunque al di sopra della soglia di accettabilità.

La varianza inter-fold risulta contenuta per tutti i modelli ($\text{std} \leq 0,0034$), indicando stabilità delle stime e assenza di dipendenza significativa dalla particolare partizione del training set.

4.3 Performance sul Test Set

La Tabella 6 confronta le performance di tutti i modelli — inclusi gli ensemble — sul test set.

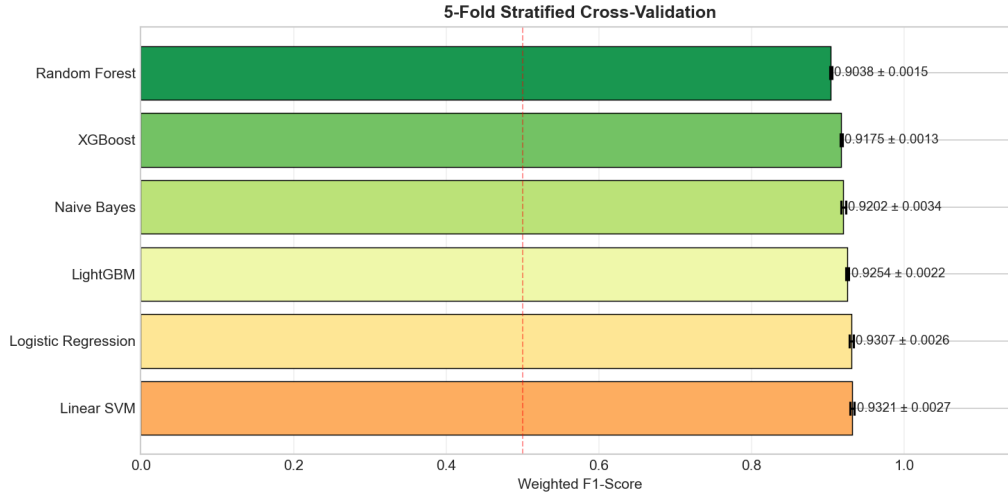


Figura 3: Risultati della 5-Fold Stratified Cross-Validation — Classificazione Binaria

Modello	Accuracy	Precision	Recall	F1
Stacking	0.9351	0.9344	0.9351	0.9345
Voting Soft	0.9342	0.9335	0.9342	0.9332
Voting Hard	0.9338	0.9331	0.9338	0.9332
Linear SVM	0.9319	0.9313	0.9319	0.9315
Logistic Regression	0.9292	0.9285	0.9292	0.9287
LightGBM	0.9246	0.9236	0.9246	0.9237
Naive Bayes	0.9228	0.9236	0.9228	0.9201
XGBoost	0.9194	0.9182	0.9194	0.9180
Random Forest	0.9042	0.9026	0.9042	0.9017

Tabella 6: Metriche sul test set — tutti i modelli, Classificazione Binaria

Gli ensemble superano sistematicamente i modelli base, con lo Stacking che raggiunge il miglior F1 complessivo (0,9345). La classifica sul test set differisce parzialmente da quella in cross-validation: il Linear SVM, migliore in CV, viene superato dallo Stacking sul test set, confermando il valore del meta-learning per la generalizzazione.

4.3.1 Modello Migliore: Stacking Ensemble

Classe	Precision	Recall	F1-Score	Support
Negative	0.8996	0.8472	0.8726	2.074
Positive	0.9467	0.9664	0.9565	5.831
Weighted Avg	0.9344	0.9351	0.9345	7.905

Tabella 7: Performance dello Stacking Ensemble per classe sul test set — Classificazione Binaria

4.3.2 Analisi della Confusion Matrix

La confusion matrix dello Stacking Ensemble evidenzia i seguenti risultati:

- **Veri Negativi (TN):** 1.757 — recensioni negative correttamente classificate (84,72%)
- **Falsi Positivi (FP):** 317 — recensioni negative erroneamente classificate come positive (15,28%)
- **Falsi Negativi (FN):** 196 — recensioni positive erroneamente classificate come negative (3,36%)
- **Veri Positivi (TP):** 5.635 — recensioni positive correttamente classificate (96,64%)

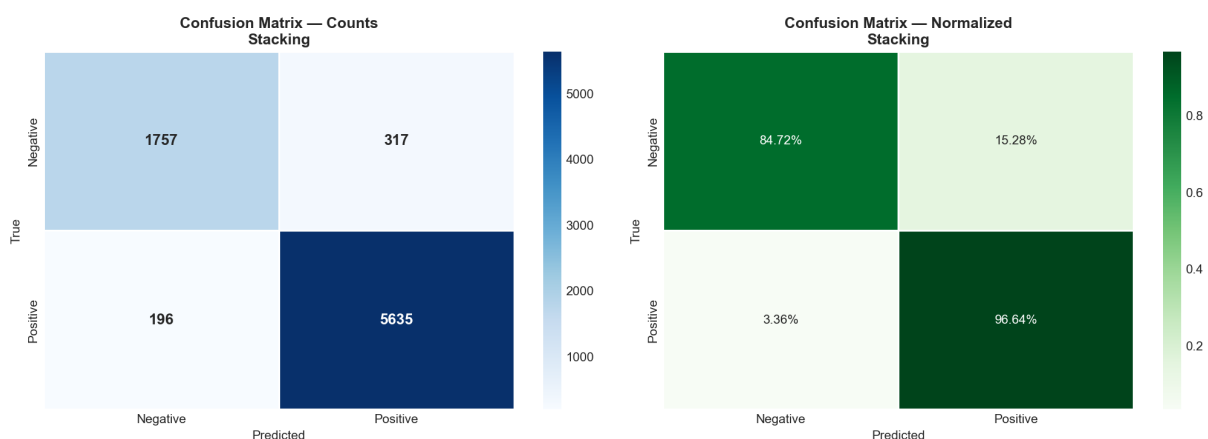


Figura 4: Confusion Matrix dello Stacking Ensemble — Classificazione Binaria

Il modello mostra un’asimmetria nei tassi di errore: il recall sulla classe positiva (96,64%) supera significativamente quello sulla classe negativa (84,72%). Questo comportamento è atteso data la maggiore rappresentazione della classe positiva nel dataset.

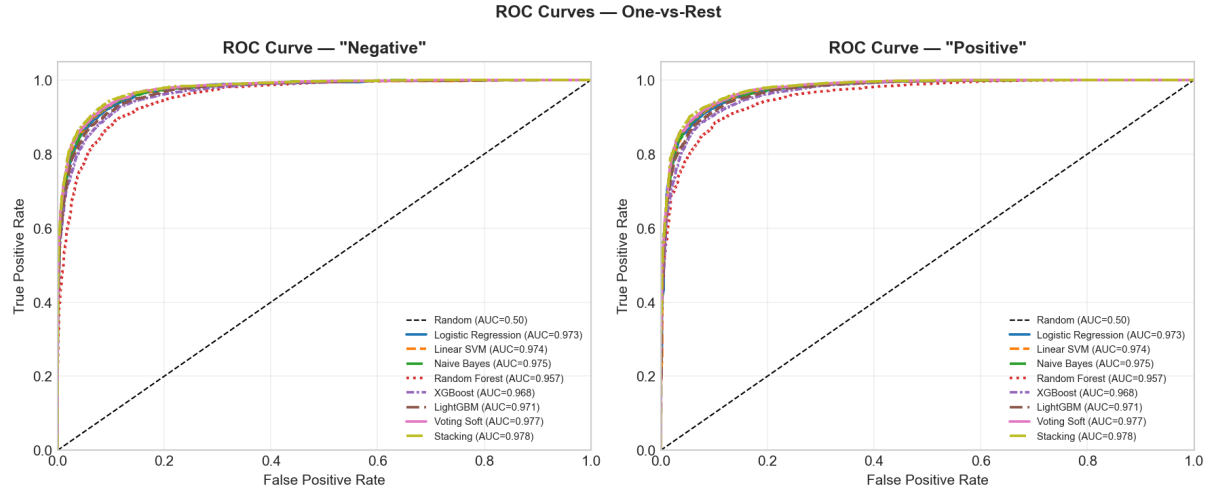


Figura 5: Curve ROC One-vs-Rest per tutti i modelli — Classificazione Binaria

Modello	AUC Negative	AUC Positive
Stacking	0.9777	0.9777
Voting Soft	0.9768	0.9768
Naive Bayes	0.9747	0.9747
Linear SVM	0.9742	0.9742
Logistic Regression	0.9733	0.9733
LightGBM	0.9709	0.9709
XGBoost	0.9675	0.9675
Random Forest	0.9571	0.9571

Tabella 8: AUC-ROC one-vs-rest per tutti i modelli — Classificazione Binaria

4.3.3 Curve ROC e AUC

I valori di AUC sono compresi tra 0,957 (Random Forest) e 0,978 (Stacking). La simmetria tra AUC Negative e AUC Positive è caratteristica del contesto binario one-vs-rest e conferma la robustezza della discriminazione in entrambe le direzioni.

4.3.4 Curve Precision-Recall

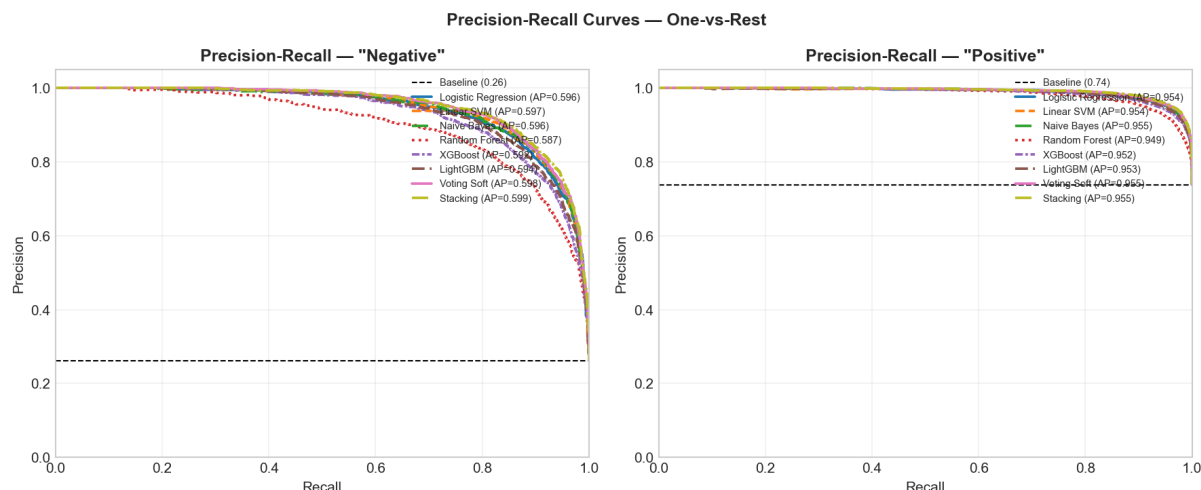


Figura 6: Curve Precision-Recall One-vs-Rest per tutti i modelli — Classificazione Binaria

Le curve Precision-Recall rivelano una netta differenza tra le due classi in termini di Average Precision (AP): la classe Positive ottiene AP compresa tra 0,949 e 0,955, mentre la classe Negative presenta AP tra 0,587 e 0,599 — significativamente superiore alla baseline (0,26) ma con margini di miglioramento più ampi, riflettendo la difficoltà intrinseca della classificazione della classe minoritaria.

4.4 Analisi delle Feature più Discriminanti

L'interpretabilità del modello Logistic Regression consente di analizzare i coefficienti TF-IDF più rilevanti per ciascuna classe. Per la classe Negative, il termine con il coefficiente più elevato è *worst* (peso $\approx 12,5$), seguito da *avoid*, *rude*, *awful* e *condescending*. Per la classe Positiva, *amazing* domina con peso $\approx 17,5$, seguito da *great*, *awesome*, *loved* e *helped*. Emergono anche bigrammi come *best professor* e *great professor*, che catturano espressioni composite ad alto potere discriminante.

4.5 Curve di Apprendimento

Per i modelli lineari (Logistic Regression e Linear SVM), il gap tra training score ($\approx 0,97$ – $0,99$) e validation score ($\approx 0,93$) si riduce progressivamente all'aumentare dei dati, indicando una lieve condizione di overfitting residuo. Random Forest, XGBoost e LightGBM mostrano un training score prossimo a 1,0 con un gap pronunciato rispetto alla validation curve ($\approx 0,87$ – $0,91$), comportamento tipico degli ensemble di alberi in spazi ad alta dimensionalità. Naive Bayes presenta un andamento peculiare: il training score decresce all'aumentare dei dati convergendo verso la validation curve, comportamento caratteristico di modelli con forte bias.

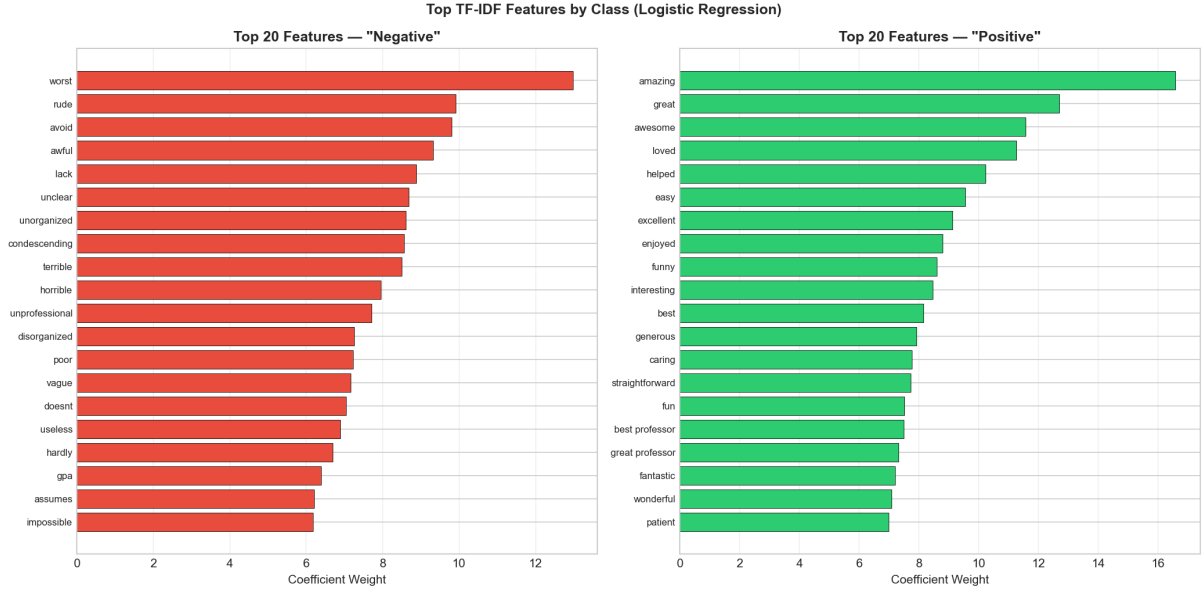


Figura 7: Top 20 feature TF-IDF per classe (Logistic Regression) — Classificazione Binaria

5 Esperimento 2: Classificazione Multi-classe

Questo capitolo presenta i risultati dell'esperimento di classificazione ternaria, in cui l'obiettivo è distinguere tra recensioni negative, neutre e positive. La classe neutra viene generata automaticamente utilizzando il punteggio di polarità calcolato da TextBlob, suddividendo il continuum del sentiment in tre fasce mediante soglie quantiliari.

5.1 Costruzione delle Etichette Ternarie

La polarità di ciascuna recensione viene calcolata tramite `TextBlob.sentiment.polarity`, che restituisce un valore continuo nell'intervallo $[-1, +1]$. Le etichette ternarie sono assegnate in base ai quantili Q_{30} e Q_{70} della distribuzione di polarità:

$$\text{label} = \begin{cases} \text{Negative} & \text{se } \text{polarity} < Q_{30} \\ \text{Neutral} & \text{se } Q_{30} \leq \text{polarity} \leq Q_{70} \\ \text{Positive} & \text{se } \text{polarity} > Q_{70} \end{cases}$$

Questa strategia garantisce per costruzione che le tre classi siano approssimativamente bilanciate (circa 30%/40%/30%).

5.2 Distribuzione delle Classi

Il dataset risulta sostanzialmente bilanciato, con la classe Neutral leggermente sovrarappresentata. Non è stata applicata nessuna strategia di over/undersampling; per compensare l'asimmetria residua, si sono utilizzati class weights differenziati (Negative: 3,0, Neutral: 2,0, Positive: 1,0) nei modelli che lo supportano.

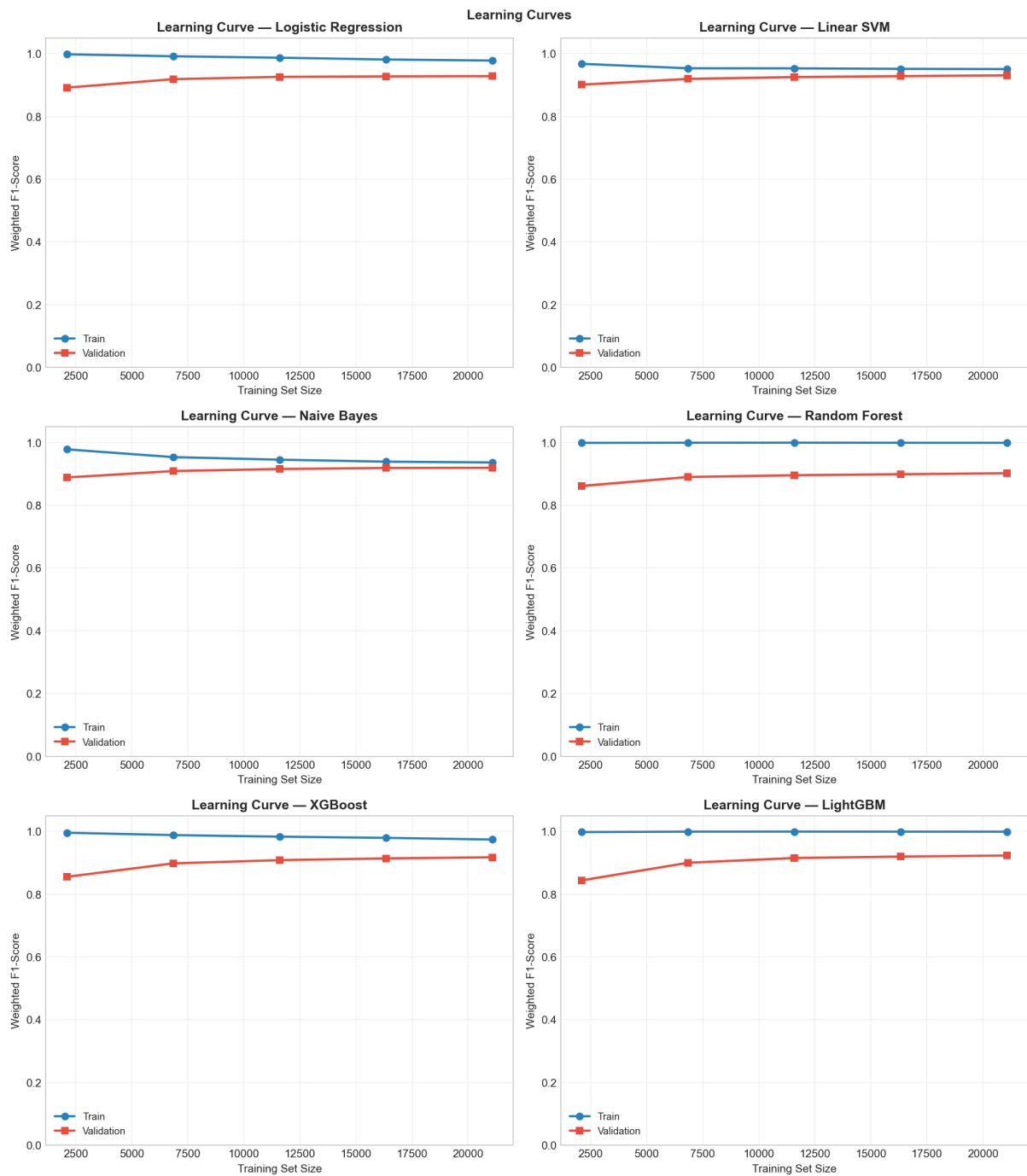


Figura 8: Curve di apprendimento per tutti i modelli — Classificazione Binaria

Classe	Campioni (train)	Percentuale
Negative	9.461	29,9%
Neutral	12.685	40,1%
Positive	9.471	30,0%
Totale	31.617	100%

Tabella 9: Distribuzione delle classi nel training set — Classificazione Ternaria

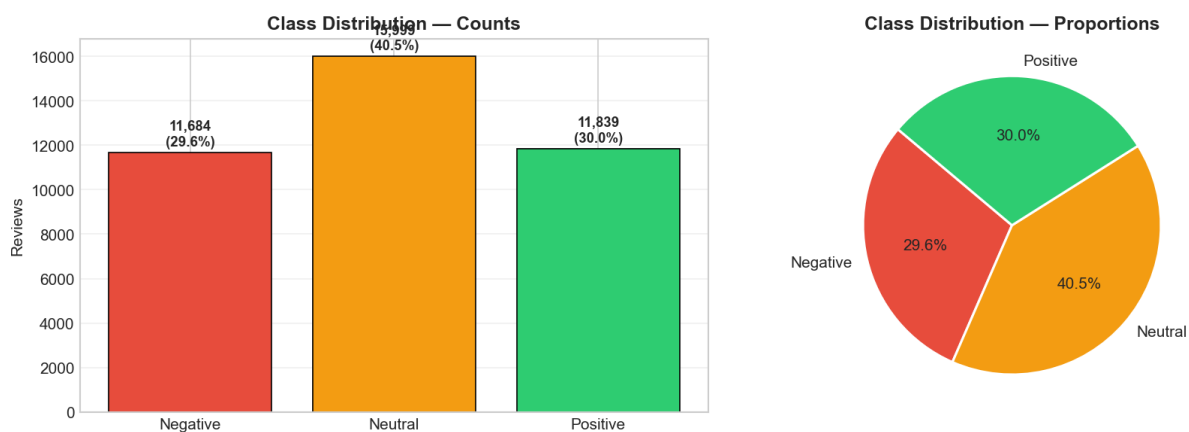


Figura 9: Distribuzione delle classi nel dataset — Classificazione Ternaria

Modello	F1-Score (weighted)	Std
Logistic Regression	0.7986	± 0.0032
LightGBM	0.7710	± 0.0034
Linear SVM	0.7633	± 0.0028
XGBoost	0.7613	± 0.0028
Random Forest	0.7042	± 0.0055
Naive Bayes	0.6783	± 0.0046

Tabella 10: Risultati della 5-Fold Stratified Cross-Validation — Classificazione Ternaria

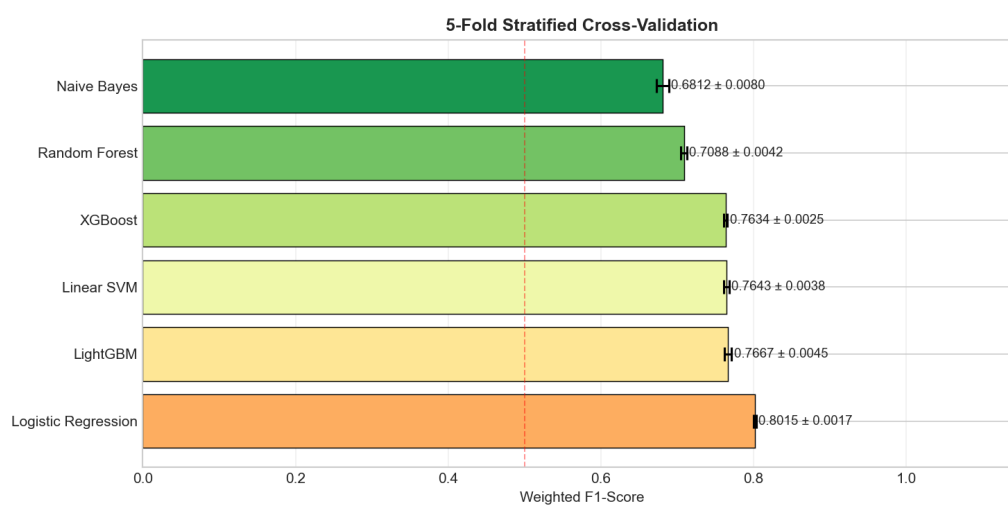


Figura 10: Risultati della 5-Fold Stratified Cross-Validation — Classificazione Ternaria

5.3 Risultati della Cross-Validation

I risultati in cross-validation mostrano un rovesciamento rispetto al caso binario: la Logistic Regression è ora il modello migliore in assoluto ($F1 = 0,7986$), mentre nel task binario era seconda dietro al Linear SVM. LightGBM guadagna posizioni (secondo con 0,7710), mentre il Naive Bayes risulta il modello meno competitivo ($F1 = 0,6783$). La deviazione standard è leggermente più alta rispetto al caso binario (fino a 0,0055), riflettendo la maggiore difficoltà del task a tre classi.

5.4 Performance sul Test Set

5.4.1 Tutti i Modelli

Modello	Accuracy	Precision	Recall	F1
Stacking	0.8190	0.8191	0.8190	0.8190
Logistic Regression	0.8143	0.8136	0.8143	0.8131
Voting Soft	0.7910	0.7941	0.7910	0.7916
Voting Hard	0.7795	0.7841	0.7795	0.7795
LightGBM	0.7755	0.7818	0.7755	0.7761
Linear SVM	0.7672	0.7671	0.7672	0.7671
XGBoost	0.7629	0.7655	0.7629	0.7635
Random Forest	0.7116	0.7146	0.7116	0.7121
Naive Bayes	0.6810	0.6989	0.6810	0.6818

Tabella 11: Metriche sul test set — tutti i modelli, Classificazione Ternaria

Lo Stacking si conferma il modello migliore anche in questo esperimento ($F1 = 0,8190$). Il gap tra i primi due modelli e il resto è più marcato che nel caso binario, suggerendo che la complessità aggiuntiva del task a tre classi avvantaggia maggiormente gli approcci ensemble.

5.4.2 Modello Migliore: Stacking Ensemble — Metriche per Classe

Classe	Precision	Recall	F1-Score	Support
Negative	0.8525	0.8651	0.8588	2.365
Neutral	0.7781	0.7837	0.7809	3.172
Positive	0.8407	0.8201	0.8303	2.368
Weighted Avg	0.8191	0.8190	0.8190	7.905

Tabella 12: Performance dello Stacking Ensemble per classe sul test set — Classificazione Ternaria

La classe Neutral è quella con le performance più basse ($F1 = 0,7809$), risultato atteso data la sua natura intrinsecamente ambigua. Le classi Negative e Positive mostrano invece F1 rispettivamente di 0,8588 e 0,8303.

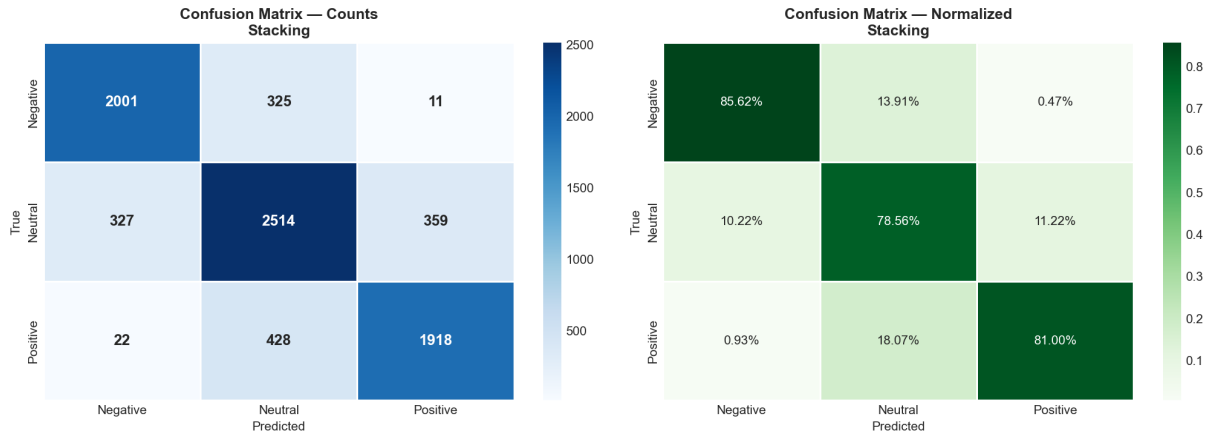


Figura 11: Confusion Matrix dello Stacking Ensemble — Classificazione Ternaria

Vera \ Predetta	Negative	Neutral	Positive
Negative	2.046 (86,51%)	306 (12,94%)	13 (0,55%)
Neutral	331 (10,44%)	2.486 (78,37%)	355 (11,19%)
Positive	23 (0,97%)	403 (17,02%)	1.942 (82,01%)

Tabella 13: Confusion Matrix — Stacking Ensemble, Classificazione Ternaria

5.4.3 Analisi della Confusion Matrix

L'analisi della confusion matrix rivela un pattern molto significativo: gli errori avvengono quasi esclusivamente tra classi *adiacenti*. Le confusioni Negative↔Positive sono praticamente assenti (solo 0,55% e 0,97%), mentre gli errori principali coinvolgono la classe Neutral. La quasi assenza di errori cross-estremo è un forte indicatore di qualità del modello.

5.4.4 Curve ROC e AUC

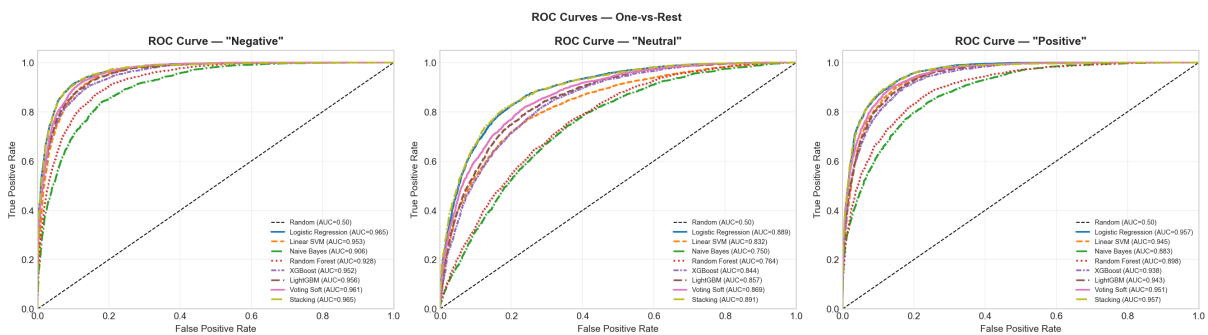


Figura 12: Curve ROC One-vs-Rest per tutti i modelli — Classificazione Ternaria

I valori di AUC confermano il pattern osservato: la classe Neutral è strutturalmente più difficile da discriminare (AUC massima = 0,8977 per lo Stacking), mentre Negative e Positive raggiungono AUC superiori a 0,96.

Modello	AUC Negative	AUC Neutral	AUC Positive
Stacking	0.9667	0.8977	0.9572
Logistic Regression	0.9666	0.8953	0.9584
Voting Soft	0.9631	0.8742	0.9506
LightGBM	0.9584	0.8594	0.9431
Linear SVM	0.9537	0.8385	0.9443
XGBoost	0.9537	0.8489	0.9379
Random Forest	0.9344	0.7789	0.8971
Naive Bayes	0.9132	0.7635	0.8854

Tabella 14: AUC-ROC one-vs-rest per tutti i modelli — Classificazione Ternaria

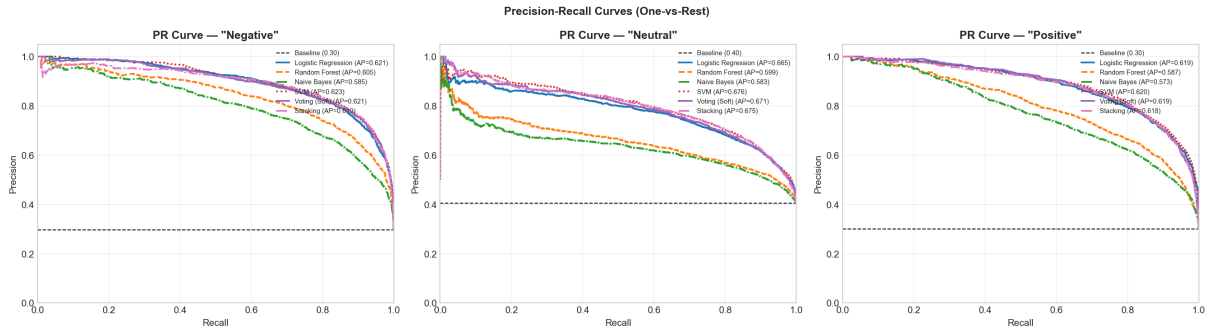


Figura 13: Curve Precision-Recall One-vs-Rest per tutti i modelli — Classificazione Ternaria

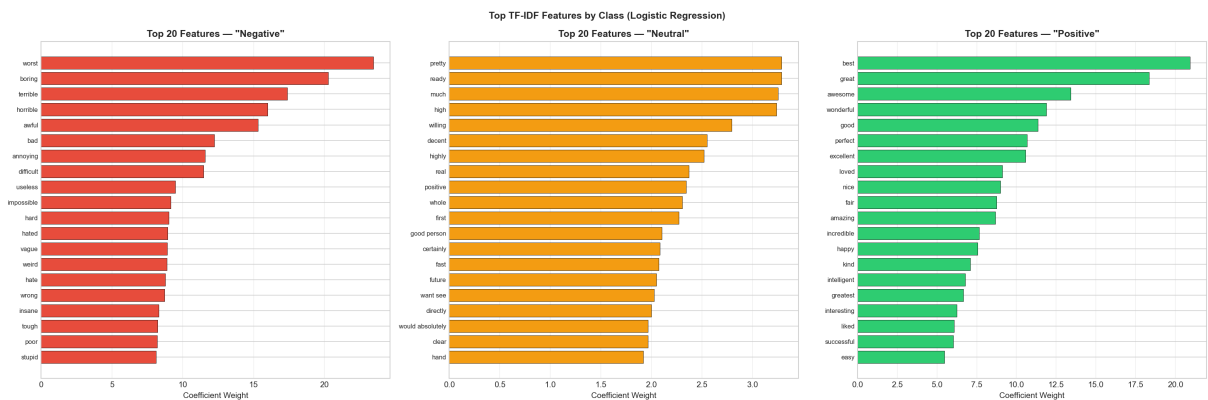


Figura 14: Top 20 feature TF-IDF per classe (Logistic Regression) — Classificazione Ternaria

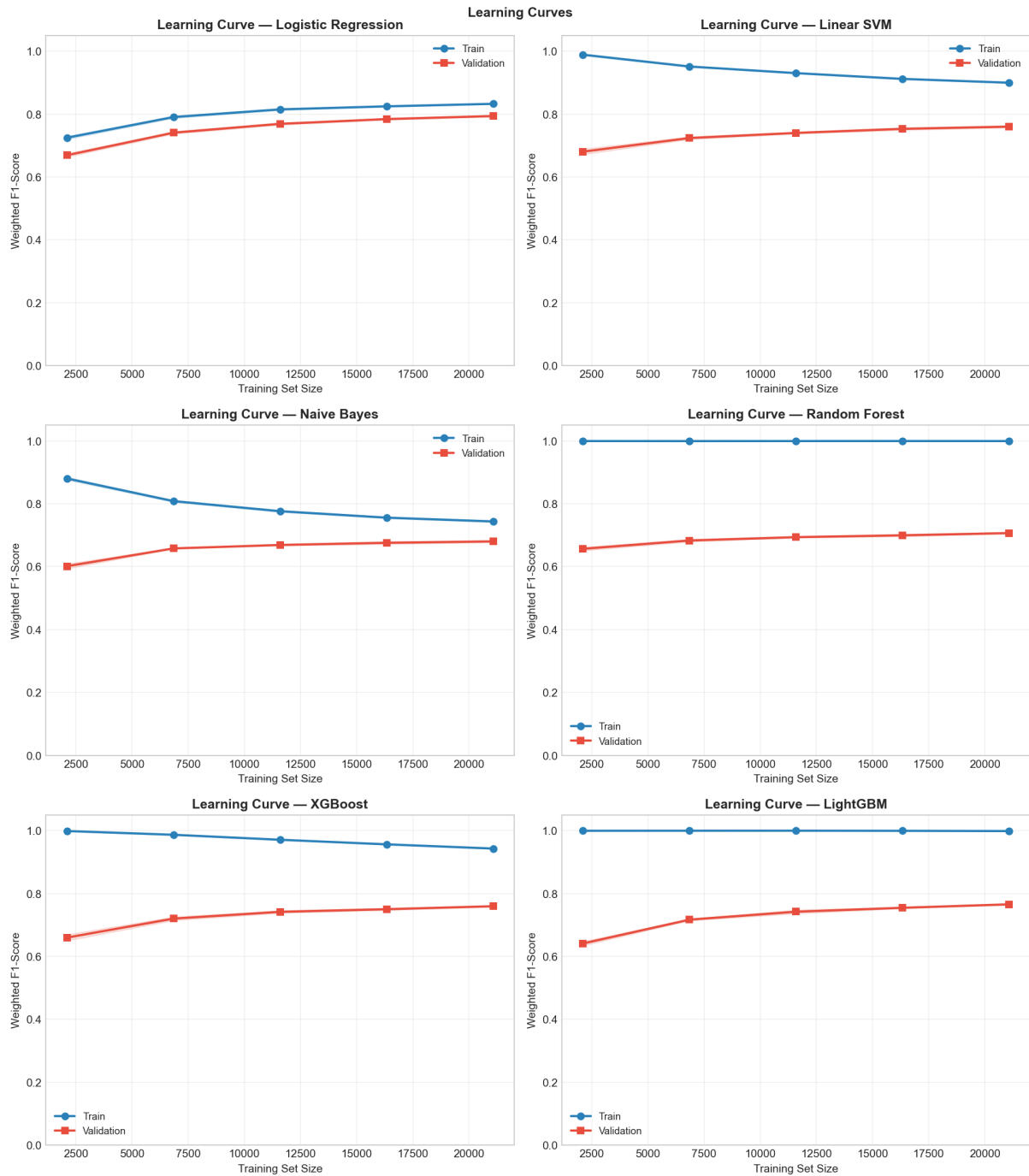


Figura 15: Curve di apprendimento per tutti i modelli — Classificazione Ternaria

6 Confronto tra Approcci

6.1 Sintesi dei Risultati

Metrica	Binario	Ternario
Classi	2	3
Modello migliore	Stacking	Stacking
Accuracy	0.9351	0.8190
Weighted Precision	0.9344	0.8191
Weighted Recall	0.9351	0.8190
Weighted F1	0.9345	0.8190
AUC-ROC (media classi)	0.9777	0.9405
Sampling strategy	None	None + class weights

Tabella 15: Confronto tra classificazione binaria e ternaria — Stacking Ensemble

6.2 Analisi Comparativa

Performance assolute. La classificazione binaria ottiene performance nettamente superiori in tutte le metriche: $F1 = 0,9345$ contro $0,8190$ del caso ternario ($\Delta = -0,115$). Questo risultato è atteso: l’aggiunta di una classe intermedia aumenta la difficoltà intrinseca del task, sia per l’ambiguità semantica del concetto di “neutro”, sia per la maggiore complessità della frontiera di decisione.

Conferma delle ipotesi di ricerca. L’ipotesi H1 è pienamente confermata: tutti i modelli superano $F1 > 0,90$ nella classificazione binaria, con il migliore che raggiunge $0,9345$. L’ipotesi H2 è supportata qualitativamente: la classificazione ternaria offre una granularità maggiore, distinguendo il sentiment ambivalente da quello polarizzato. L’ipotesi H3 è parzialmente confermata: nel task binario il Random Oversampling apporta un miglioramento marginale rispetto alla baseline senza sampling ($F1\ 0,9289$ vs $0,9281$), suggerendo che l’imbalance ratio di $2,81:1$ non costituisce una criticità grave per i modelli lineari su spazi TF-IDF.

Comportamento degli ensemble. In entrambi gli esperimenti, lo Stacking Ensemble è il modello più performante sul test set, pur non essendo necessariamente il migliore in cross-validation. Questo conferma il valore del meta-learning: il meta-learner apprende come combinare i punti di forza dei singoli classificatori, migliorando la generalizzazione anche quando i modelli base sembrano già competitivi.

Modelli lineari vs. boosting. Un risultato rilevante è il comportamento opposto dei modelli lineari rispetto ai modelli basati su alberi al variare del task. In cross-validation:

- **Task binario:** Linear SVM (1°) > Logistic Regression (2°) > LightGBM (3°)
- **Task ternario:** Logistic Regression (1°) > LightGBM (2°) > Linear SVM (3°)

Questo suggerisce che la regolarizzazione L1 della Logistic Regression nel task ternario aiuta a gestire la maggiore difficoltà della frontiera multi-classe, mentre LightGBM beneficia della sua capacità di modellare interazioni non lineari tra le feature in presenza di tre classi.

Qualità degli errori. La confusion matrix del task ternario rivela che il 99% degli errori coinvolge classi adiacenti (Neutral $\leftrightarrow$ estremi), con una confusione Negative $\leftrightarrow$ Positive inferiore all'1%. In un'applicazione reale, classificare erroneamente una recensione neutro come positiva o negativa è molto meno problematico che classificare una recensione molto negativa come positiva.

7 Conclusioni

Questo progetto ha affrontato il problema della sentiment analysis su recensioni universitarie (dataset RateMyProfessors, 39.522 campioni) confrontando due approcci: classificazione binaria (Negative/Positive) e classificazione ternaria (Negative/Neutral/Positive). L'intera pipeline — dal preprocessing TF-IDF all'ottimizzazione degli iperparametri e alla valutazione ensemble — è stata implementata in modo modulare e riproducibile in Python.

7.1 Principali Risultati

I risultati ottenuti confermano le ipotesi di ricerca e forniscono indicazioni metodologiche di interesse:

- Lo Stacking Ensemble è il modello migliore in entrambi i task, raggiungendo $F1 = 0,9345$ nel caso binario e $F1 = 0,8190$ nel caso ternario.
- I modelli lineari (Logistic Regression, Linear SVM) sono altamente competitivi su rappresentazioni TF-IDF, spesso superiori a modelli più complessi come XGBoost e Random Forest.
- La gestione delle negazioni nel preprocessing (prefisso NOT_) contribuisce alla capacità discriminativa del modello, soprattutto per la classe negativa.
- Nel task ternario, gli errori del modello sono quasi esclusivamente tra classi adiacenti, il che minimizza l'impatto pratico degli errori di classificazione.
- L'AUC-ROC rimane elevata in entrambi i task: 0,978 nel binario e 0,941 (media) nel ternario, indicando buona calibrazione probabilistica dei modelli.

7.2 Limitazioni

- **Qualità delle etichette ternarie:** le etichette della classe Neutral derivano da un punteggio automatico (TextBlob + quantili) e non da annotazioni umane. La qualità dell'etichettatura è quindi limitata dalla capacità di TextBlob di catturare la sfumatura del linguaggio accademico informale.
- **Dominio specifico:** i modelli sono addestrati su un corpus specifico (recensioni di professori universitari anglofoni). La generalizzazione ad altri domini o lingue non è garantita senza fine-tuning.
- **Rappresentazione lessicale:** l'utilizzo di TF-IDF non cattura relazioni semantiche tra termini (sinonimi, contesti d'uso). Approcci basati su word embeddings o modelli Transformer potrebbero migliorare ulteriormente le performance.

7.3 Sviluppi Futuri

Possibili estensioni del lavoro includono: l'utilizzo di rappresentazioni contestuali (BERT, RoBERTa) per catturare semantica compositiva; la validazione con annotazioni umane per la classe Neutral; l'esplorazione di tecniche di active learning per ridurre il costo di etichettatura; e l'applicazione della pipeline a dataset multi-lingua per valutarne la trasferibilità.

Riferimenti bibliografici

- [1] Jurafsky, D., & Martin, J. H. (2023). *Speech and Language Processing* (3rd ed.). Pearson.
- [2] Vaswani, A., et al. (2017). Attention is all you need. *Advances in Neural Information Processing Systems*, 30.
- [3] Pedregosa, F., et al. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825-2830.
- [4] Bird, S., Klein, E., & Loper, E. (2009). *Natural Language Processing with Python*. O'Reilly Media.
- [5] Chawla, N. V., et al. (2002). SMOTE: Synthetic minority over-sampling technique. *Journal of Artificial Intelligence Research*, 16, 321-357.